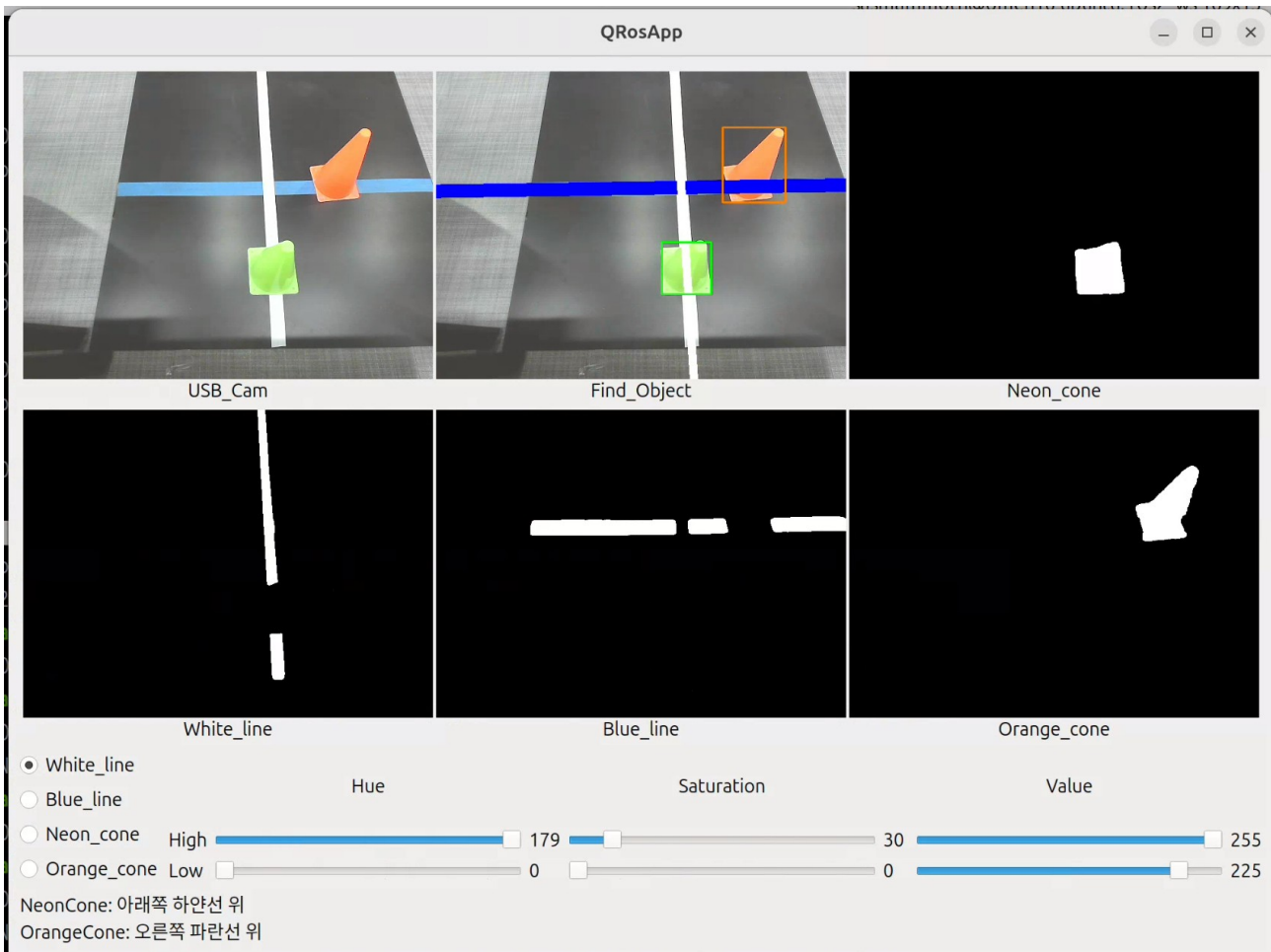


HW_2

중앙의 흰선, 푸른 선을 기준으로 콘의 위치를 식별하기.

9가지 위치 + 존재하지 않음, 총 10가지 경우로 주황색과 네온 색 콘을 판별하고, 각각의 요소를 디텍팅 한다.

실행 결과



과제에 관한 간단한 고찰

1. 과제에 관한 설명

본 과제에서 USB_CAM을 통해 이미지를 불러온다. ros2를 이용해 자동으로 이미지가 33ms에 한번 씩 불러 오게 되는데, 이때마다 My_corn_cv::image_callback 함수가 실행된다. 간단한 예외 처리를 거친 후 네온 색 콘 마스크, 오렌지 색 콘 마스크, 하얀 선 마스크, 파란 선 마스크 등을 생성하는 함수를 거친다. 각각의 함수에서는 몇 번의

시행착오를 거친 후 찾은 가장 알맞은 크기의 가우시안 블러 필터와 inRange 함수, 모폴로지 열림과 모폴로지 닫힘 함수를 통과시켜 각 요소에 대한 마스크를 구한다.

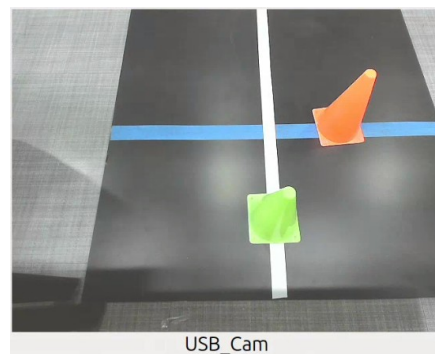
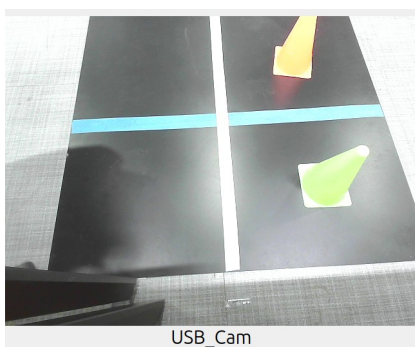
위에서 구한 4개의 마스크를 가지고 find_object_func를 실행시킨다. 이 함수에서는 하얀 선과 파란 선 마스크, cv::fitLine 함수를 이용해 점들의 집합에 가장 맞는 직선을 구한다. cv::minAreaRect 함수 등을 이용해 길이, 두께를 구하고 직선을 화면에 그린다. 다음으로 각각의 콘(마스크) 마다 처리를 하는데, cv::findContours를 이용해 마스크의 하얀 부분에서 외각 선을 탄다. 외각 선 객체를 이용해 넓이가 500 이상인 것만 체크하고 그중 가장 큰 객체를 콘이라 생각한다. 만약 넓이가 500이 넘는 것이 없으면 콘이 존재하지 않는다고 생각한다. 이 외각 선에 콘 색깔에 맞는 네모 박스를 그리고 콘의 위치를 판별한다. 직선 2개는 무조건 파란색이 x축, 하얀색이 y축이라 가정을 한다. 콘의 가장자리 네모 박스의 위치와 직선 2개간의 위치를 이용해 선이 박스를 관통하면 선 위에 존재함, 아니면 1,2,3,4분 면 등에 존재한다 가정하였다. 위 연산을 통해 콘의 위치 문자열을 만든다.

find_object_func와 기타 마스크 생성 함수를 이용해 만든 출력 문자열과 cv::Mat 형식의 화면 배열을 콜백등으로 qnode 객체로 던지면, qnode에서 받아서 ui에서 만들어 놓은 각각에 맞는 QLabel에 문자열 형식과 이미지 형식으로 전시한다.

2. 과제에 관한 고찰

본 과제에서 각각의 색깔 콘과 색깔 선을 판단할 때 가우시안 블러 필터 등의 전처리나 모폴로지 열림-닫힘 등의 후처리를 거친 하였지만, 최종적으로는 HSV값을 이용해 특정 범위 내의 색 영역만 추출하는 inRange 함수를 이용하여 각각의 물체를 판별하였다. 이때 최적의 값/세팅을 찾느라 여러 번의 시행착오와 AI의 도움 등을 받았다.

가장 먼저 카메라의 빛 번짐이 너무 심하여 아무리 임계값을 조정하여도 꼬깔의 반사광등이 하얀선으로 인식되는 문제가 있었다. 따라서 먼저 리눅스에 내장된 카메라등을 다루는 도구인 v4l2-ctl를 이용해 카메라 노출을 수동으로 조절하여 빛 번짐을 줄였다.



(좌측은 자동 노출, 우측은 설정값을 건드린 수동 노출 이다.)

그 후 최적의 HSV 임계값을 찾으려고 여러번 시도 하였다. HSV값에서 S와 V는 조명 변화에 민감하여 임계값 바운더리가 크고, H는 각 색을 찾아야 하므로 임계값 바운더리가 작다. (하얀 선은 H보다는 S와 V가 중요하므로 H는 [0, 179] 이고, S와 V는 각각 [0, 30], [255, 255]이다.) 따라서 하얀 선을 제외한 3개의 HSV 바운더리는 H : 30~50, S : 195~205, V : 135 이다.

표 1: 경험적으로 찾은 임계값

```
set_hsv_range(Target::WhiteLine, cv::Scalar(0, 0, 225), cv::Scalar(179, 30, 255)); // 하얀선
set_hsv_range(Target::BlueLine, cv::Scalar(90, 50, 120), cv::Scalar(120, 255, 255)); // 파란선
set_hsv_range(Target::OrangeCone, cv::Scalar(10, 50, 120), cv::Scalar(35, 255, 255)); // 오렌지콘
set_hsv_range(Target::NeonCone, cv::Scalar(35, 60, 120), cv::Scalar(85, 255, 255)); // 네온콘
```

위의 값들은 여러번의 시행착오를 거쳐 연구실의 조명/색깔콘 등을 경험적으로 타이트하게 조정한 거라 게임장을 조금만 이동하는 등의 약간의 조명의 밝기, 그림자의 변화 콘과 경기장의 색깔 변화가 일어나면 제대로 동작하지 않을 가능성이 많이 존재한다. 또한 아래의 사진과 같이 사람이 콘을 수동으로 움직이면서 입은 옷의 색이 파란 선의 색과 거의 같아서 cv::inRange 등의 함수 만으로는 정확한 검출이 어렵다.

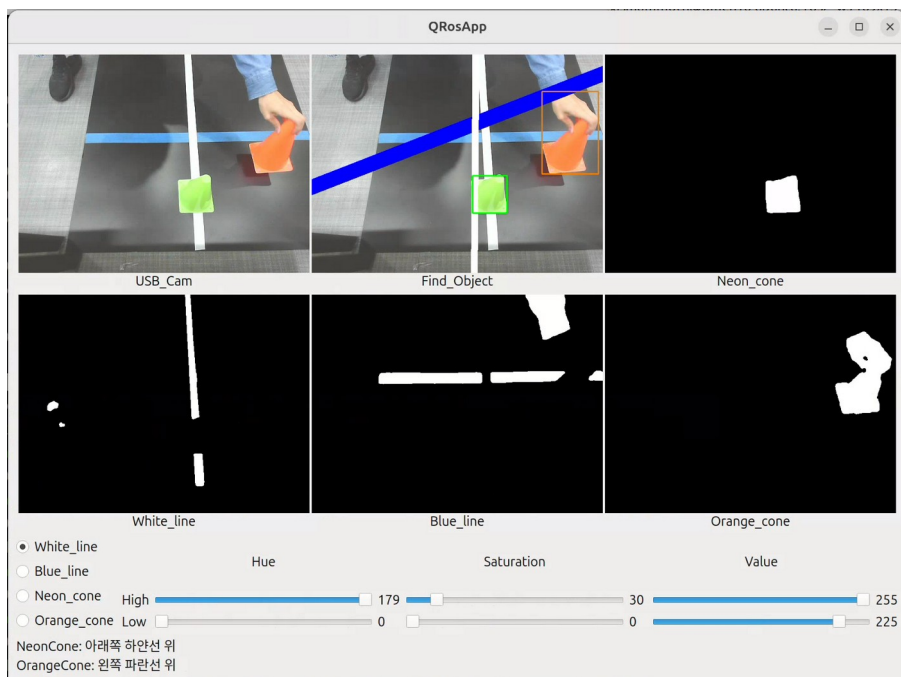


그림 1: 청 자켓을 입고 있어 소매가 파란선으로 인식되기 때문에 파란 선에 큰 왜곡이 생김

따라서 실제 대회 / 실무 등에서는 임계값을 설정할 때 내가 했던 주먹구구식이 아니라 다양한 방법을 사용한다. 위와 같은 ui를 만들어 화면을 보고 마우스 등으로 물체를 지정하여 그 물체를 구성하는 픽셀의 HSV 임계값을 평균-표준편차 등 통계적인 방식으로 구하거나, 조명등의 변화에 따라 S, V 임계값을 동적으로 실시간으로 할당하는 방식을 사용한다고 한다. 최종적으로는 딥러닝을 이용해서 색이 아니라 물체의 형상을 인식하는 것이 가장 좋다고 한다.

OpenCV에 관한 개념

OpenCV는 이미지와 영상을 처리하는 오픈소스 컴퓨터비전 라이브러리이다. RGB2HSV등 간단한 표시 형식 변화 부터 화면 크기 조정, 도형/글자 넣기, 객체인식과 물체추적등 매우 다양한 라이브러리를 제공하고 Yolo, TensorFlow등의 AI와 결합하여 많은 것들을 할 수 있다.

이런 다양한 것들 중에서 강의를 들으면서 든 생각인 ‘픽셀의 정보 RGB-HSV’에 관한 고찰과 검색한 사실 등을 작성하겠다.

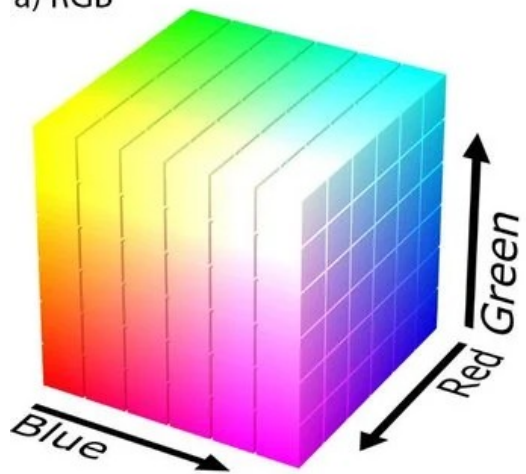
1. 픽셀과 HSV, RGB와 HSV의

이미지나 비디오 화면은 픽셀이 2560x1440등 2차원으로 배열되어 있지만 화면에 표시하는 색이 컬러이면, 각 픽셀당 3개(인쇄등에서 사용되는 CMYK는 4개이다)의 값을 가지므로 3차원 형태의 객체에 값이 저장된다. 이 픽셀의 집합을 잘 연산하기 위해서 OpenCV에서는 cv::Mat이라는 객체를 이용하여 연산을 한다.

RGB는 빛의 삼원색인 rgb를 이용하여 표시한 것으로 다양한 색깔을 표현하는 것이 가능하고 (보통 하나의 채널에 8bit를 할당하므로 $2^8 * 2^8 * 2^8 = 16777216$ 개의 색을 표시 가능하다) 사람 눈에 있는 광수용체 와 같은 형식이고 디스플레이 등에 사용되는 sRGB와 호환이 잘 되므로 많이 사용된다.

HSV는 색조 - 채도 - 명도를 이용해서 빛을 나타내는 형식이고 RGB보다 사람에게 더 직관적으로 사용이 가능 하다는 장점이 있다. RGB와 HSV에서 H를 제외한 나머지 모든 부분은 0~255(8bit)의 값을 사용하는데 H 값만 0~179값을 사용한다. H는 원래 0~359의 값을 갖는 게 보통이지만, 아마 0~359는 8bit안에 전부 안 들어가서 비트를 하나 더 할당해야 하므로 빠른 연산 / 데이터 형식의의 깔맞춤 을 위해 이와 같이 한것이 아닌가 하는 생각이 들었다.

a) RGB



b) HSV

